Selenium WebDriver with Python

A Complete Reference Guide for Software Testers

From First Script to Advanced Automation Frameworks

Intended Audience:

New joiners to the QA/Test Engineering team (no prior Selenium experience required)
Experienced testers seeking a structured reference and best-practice cheat sheet

Covers: WebDriver fundamentals · Locators · Waits · ActionChains · Frames & Windows · JavaScript Executor · Page Object Model · Pytest Integration · Exception Handling · Best Practices · Cheat Sheet

Document Version 1.0

Contents

1 Introduction to Selenium and WebDriver **3**
- 1.1 What is Selenium? . . . 3
- 1.2 Why use Selenium for testing? . . . 3
- 1.3 Selenium 4 and Selenium Manager . . . 4
- 1.4 Installing the environment . . . 4
  - 1.4.1 Prerequisites . . . 4
  - 1.4.2 Installing the Selenium package . . . 4
  - 1.4.3 Your first script . . . 5

2 The WebDriver Object in Depth **6**
- 2.1 Creating driver sessions for different browsers . . . 6
- 2.2 Browser Options and Capabilities . . . 6
- 2.3 Navigation commands . . . 6
- 2.4 Reading browser / page state . . . 7
- 2.5 Window and tab management . . . 7
- 2.6 Managing timeouts . . . 8

3 Locating Elements **9**
- 3.1 The `By` class and locator strategies . . . 9
  - 3.1.1 Locator strategies (the `By` class) . . . 9
- 3.2 CSS Selector cheat sheet . . . 10
- 3.3 XPath cheat sheet . . . 10
- 3.4 Chaining locators (finding elements inside elements) . . . 11
- 3.5 Inspecting a `WebElement` . . . 11

4 Waiting Strategies **12**
- 4.1 Why waits matter . . . 12
- 4.2 Hard-coded sleeps (avoid) . . . 12
- 4.3 Implicit waits . . . 12
- 4.4 Explicit waits – `WebDriverWait` . . . 13
- 4.5 Common `expected_conditions` (EC) . . . 13
- 4.6 Writing a custom wait condition . . . 14
- 4.7 Fluent Wait (fine-grained control) . . . 14

5 Interacting with Elements **15**
- 5.1 Basic interaction methods . . . 15
- 5.2 Sending special keys . . . 15
- 5.3 Working with dropdowns – the `Select` class . . . 16
- 5.4 Checkboxes and radio buttons . . . 16
- 5.5 File uploads . . . 16

6 ActionChains: Advanced Mouse and Keyboard Control **18**
6.1 What is `ActionChains`? . . . 18
6.2 Hovering (mouse-over menus) . . . 18
6.3 Drag and drop . . . 19
6.4 Right-click (context menu) and double-click . . . 19
6.5 Keyboard modifiers combined with mouse actions . . . 19
6.6 Full `ActionChains` method reference . . . 20

7 Frames, Alerts, and Multi-Window Handling **21**
7.1 IFrames . . . 21
7.2 JavaScript alerts, confirms, and prompts . . . 21
7.3 Multiple windows and tabs – revisited . . . 22

8 JavaScript Execution, Screenshots, and Cookies **23**
8.1 Executing JavaScript directly . . . 23
8.2 Asynchronous scripts . . . 24
8.3 Taking screenshots . . . 24
8.4 Cookies . . . 24

9 Exceptions and Robust Error Handling **25**
9.1 Common exceptions you will meet . . . 25
9.2 Import path for exceptions . . . 25
9.3 Retry pattern for stale elements . . . 26
9.4 Handling intercepted clicks . . . 26

10 Structuring Test Code: The Page Object Model **27**
10.1 The problem POM solves . . . 27
10.2 A basic Page Object . . . 27
10.3 Using the Page Object from a test . . . 28
10.4 Base Page pattern (shared behaviour) . . . 28

11 Integrating with Pytest **30**
11.1 Why pytest over raw scripts or unittest . . . 30
11.2 The driver fixture . . . 30
11.3 Fixture scope . . . 31
11.4 Automatic screenshot-on-failure . . . 31
11.5 Parametrizing tests (e.g. cross-browser or data-driven runs) . . . 31
11.6 Running and reporting . . . 32

12 Best Practices and Common Pitfalls **33**
12.1 Locator strategy . . . 33
12.2 Synchronization . . . 33
12.3 Test design . . . 33
12.4 Performance and stability . . . 33
12.5 Maintainability . . . 34

13 Quick Reference Cheat Sheet **35**
13.1 Setup . . . 35
13.2 One-page summary table . . . 35
13.3 Most-used `expected_conditions` . . . 36
13.4 Glossary . . . 36

Chapter 1

Introduction to Selenium and WebDriver

1.1 What is Selenium?

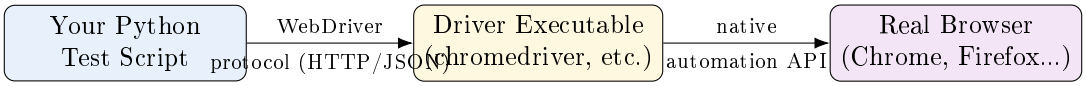
Selenium is an open-source umbrella project for browser automation. It lets you write code that opens a real browser, clicks buttons, fills in forms, reads text, and verifies that a web application behaves correctly – exactly what a human tester would do, but repeatable, fast, and scriptable.

Selenium is not a single tool; it is a family of tools:

- **Selenium WebDriver** – the core API used to drive a real browser programmatically. This is what this guide focuses on.
- **Selenium IDE** – a browser extension for record-and-playback test creation. Good for quick prototyping, not for serious automation frameworks.
- **Selenium Grid** – infrastructure for running tests in parallel across many machines/browsers (useful for cross-browser and scale testing).

CONCEPT

What is WebDriver, precisely?
WebDriver is a *W3C standard protocol* that defines how an external program (your Python script) can control a browser. Each browser vendor (Google, Mozilla, Microsoft) ships a small executable called a **driver** (e.g. `chromedriver`, `geckodriver`, `msedgedriver`) that translates WebDriver protocol commands into native browser instructions.
In plain terms: your Python code talks to the driver executable over HTTP (a local REST-like API), and the driver talks to the browser directly. This is why installing the matching driver binary is a prerequisite for automation to work.



1.2 Why use Selenium for testing?

- Simulates real user behaviour in a real browser (unlike API tests, it exercises the full stack: HTML, CSS, JavaScript, rendering).
- Supports all major browsers: Chrome, Firefox, Edge, Safari.
- Language bindings exist for Python, Java, C#, JavaScript, Ruby, Kotlin.
- Integrates with test runners (`pytest`, `unittest`, `robot framework`) and CI/CD pipelines.

- Free and open source, with a large community and ecosystem (Selenium Grid, Selenium Manager, browser extensions).

NOTE

Selenium automates the *browser*, not the operating system. It cannot natively handle OS-level dialogs (e.g. a Windows "Save File" dialog) or desktop applications. Those require companion tools like `pyautogui` or OS-specific automation libraries.

1.3 Selenium 4 and Selenium Manager

Selenium 4 (the current major version) introduced **Selenium Manager**, which automatically detects your installed browser version and downloads the matching driver executable for you. This removes most of the historical pain of manually downloading and placing driver binaries on your `PATH`.

TIP

As of Selenium 4.6+, in most cases you do **not** need to manually download `chromedriver` or `geckodriver`. Simply instantiate `webdriver.Chrome()` and Selenium Manager handles the rest. Manual driver management is still useful for pinning exact driver versions in CI environments.

1.4 Installing the environment

1.4.1 Prerequisites

- Python 3.9 or later (3.11+ recommended).
- A supported browser installed (Google Chrome is the most common choice for automation).
- `pip` package manager.

1.4.2 Installing the Selenium package

```
# Create and activate a virtual environment (recommended)
python -m venv venv
source venv/bin/activate     # on Linux/macOS
venv\Scripts\activate        # on Windows

# Install Selenium
pip install selenium

# (Optional but recommended for scalable test suites)
pip install pytest pytest-html webdriver-manager
```

Package	Purpose
`selenium`	Core WebDriver bindings for Python.
`pytest`	Popular, flexible Python test runner (recommended over `unittest` for new projects).
`pytest-html`	Generates HTML test reports.
`webdriver-manager`	Alternative driver-management utility (useful if you don't want to rely on Selenium Manager).

1.4.3 Your first script

```python
from selenium import webdriver
from selenium.webdriver.common.by import By

# 1. Launch a Chrome browser session
driver = webdriver.Chrome()

# 2. Navigate to a URL
driver.get("https://example.com")

# 3. Locate an element and read its text
heading = driver.find_element(By.TAG_NAME, "h1")
print(heading.text)        # -> "Example Domain"

# 4. Always close the browser session when done
driver.quit()
```

RETURNS

`webdriver.Chrome()` returns a `WebDriver` object (specifically a `selenium.webdriver.chrome.webdriver.WebDriver` instance). This object represents **one browser session** and is the entry point for every subsequent command: navigation, element lookup, window management, cookies, JavaScript execution, and screenshots.

COMMON PITFALL

Forgetting to call `driver.quit()` leaves orphaned browser and driver processes running in the background. Over a long test suite run, this leaks memory and can eventually crash your CI runner. Always pair driver creation with a teardown step (see Chapter 11 on `pytest` fixtures for a robust pattern).

Chapter 2

The WebDriver Object in Depth

2.1 Creating driver sessions for different browsers

```
from selenium import webdriver

chrome_driver = webdriver.Chrome()
firefox_driver = webdriver.Firefox()
edge_driver   = webdriver.Edge()
safari_driver = webdriver.Safari() # macOS only, needs manual enabling
```

2.2 Browser Options and Capabilities

Options objects let you configure how the browser launches: headless mode, window size, custom user agents, download preferences, proxy settings, and more.

```
from selenium import webdriver
from selenium.webdriver.chrome.options import Options

options = Options()
options.add_argument("--headless=new")   # run with no visible UI
options.add_argument("--window-size=1920,1080")
options.add_argument("--disable-gpu")
options.add_argument("--start-maximized")
options.add_experimental_option("excludeSwitches", ["enable-automation"])

driver = webdriver.Chrome(options=options)
```

CONCEPT

Headless mode runs the browser without rendering a visible window. It is faster and lighter – ideal for CI/CD pipelines – but a few behaviours (e.g. certain screen-size-dependent CSS, some file dialogs) can differ slightly from a headed browser. Always run a final validation pass in headed mode before trusting a brand-new suite fully.

2.3 Navigation commands

```
driver.get("https://example.com") # navigate to a URL (waits for page load)
```

```
driver.back()                    # browser back button
driver.forward()                 # browser forward button
driver.refresh()                 # reload current page
```

RETURNS

None of the navigation methods above return a meaningful value (they return `None`). Their effect is a *side effect*: the browser's loaded document changes. You verify success by asserting on subsequent state (URL, title, or element presence), not on a return value.

2.4 Reading browser / page state

```
driver.title          # -> str: the <title> of the current page
driver.current_url    # -> str: the URL currently loaded
driver.page_source     # -> str: full raw HTML of the current page
driver.name            # -> str: name of the underlying browser, e.g. "chrome"
```

Property	Return type	Typical use
`driver.title`	`str`	Assert the correct page has loaded.
`driver.current_url`	`str`	Verify redirects / routing logic.
`driver.page_source`	`str`	Debugging, or regex/text search as a last resort (prefer locators).

2.5 Window and tab management

```
driver.maximize_window()
driver.minimize_window()
driver.set_window_size(1024, 768)
driver.set_window_position(0, 0)

# Multiple tabs / windows
driver.window_handles       # -> list[str]: IDs of every open window/tab
driver.current_window_handle # -> str: ID of the window currently in focus

# Open a new tab and switch to it
driver.switch_to.new_window('tab')
driver.switch_to.window(driver.window_handles[0]) # switch back to the first tab
```

RETURNS

`driver.window_handles` returns a `list` of opaque string identifiers, one per open browser window/tab. Selenium only ever "controls" one window at a time (the one in focus); to interact with a different tab (e.g. one opened after clicking a link with `target="_blank"`), you must explicitly call `driver.switch_to.window(handle)`.

```
# Common pattern: handle a link that opens a new tab
original_window = driver.current_window_handle
link.click()                                # opens a new tab

```

```
for handle in driver.window_handles:
    if handle != original_window:
        driver.switch_to.window(handle)
        break

# ... interact with the new tab ...

driver.close()                    # closes only the CURRENT tab/window
driver.switch_to.window(original_window)
```

COMMON PITFALL

`driver.close()` closes only the currently focused window/tab. `driver.quit()` closes *all* windows opened by this driver session and ends the WebDriver process entirely, along with the underlying driver executable. Calling `close()` on the last remaining window without ever calling `quit()` can leave the driver process alive.

2.6 Managing timeouts

```
driver.set_page_load_timeout(30)  # seconds to wait for driver.get() to finish
driver.set_script_timeout(30)     # seconds to wait for async JS scripts
driver.implicitly_wait(10)        # see Chapter 4 on Waits -- use sparingly
```

Chapter 3

Locating Elements

3.1 The `By` class and locator strategies

Almost everything you do in Selenium starts with finding an element. Selenium provides two core methods on the driver (and on any `WebElement`, for finding elements nested inside it):

```
driver.find_element(by, value)  # returns ONE WebElement (the first match)
driver.find_elements(by, value) # returns a list of WebElement objects
```

RETURNS

- `find_element` returns a single `WebElement` object, or raises `NoSuchElementException` if nothing matches.
- `find_elements` **never raises** for zero matches – it returns an empty `list`, `[]`. This makes it the safe choice when you want to check for the presence/absence/count of elements without wrapping code in a try/except.

3.1.1 Locator strategies (the `By` class)

```
from selenium.webdriver.common.by import By

driver.find_element(By.ID, "username")
driver.find_element(By.NAME, "email")
driver.find_element(By.CLASS_NAME, "btn-primary")
driver.find_element(By.TAG_NAME, "input")
driver.find_element(By.LINK_TEXT, "Sign up")
driver.find_element(By.PARTIAL_LINK_TEXT, "Sign")
driver.find_element(By.CSS_SELECTOR, "div.card > a.btn")
driver.find_element(By.XPATH, "//button[text()='Submit']")
```

Strategy	When to use it	Speed
`ID`	Best choice whenever an element has a unique, stable `id` attribute.	Fastest
`NAME`	Good for form fields lacking an id but with a `name` attribute.	Fast
`CSS_SELECTOR`	Flexible, readable, fast. The recommended default for most cases.	Fast
`XPATH`	Needed for text-based matching, traversing *upward* (parents/ancestors), or complex conditions.	Slightly slower
`CLASS_NAME`	Only when the class uniquely identifies the element (careful: many elements often share classes).	Fast
`LINK_TEXT` / `PARTIAL_LINK_TEXT`	Only applies to `<a>` anchor tags.	Fast
`TAG_NAME`	Rarely unique by itself; mostly used with `find_elements` to grab all elements of a type.	Fast

TIP

Prefer `ID` > `CSS_SELECTOR` > `XPATH` in that order of preference, unless XPath's extra power (text matching, axis navigation like "parent" or "following-sibling") is genuinely required. CSS selectors are generally faster to evaluate and easier for teammates to read than complex XPath expressions.

3.2 CSS Selector cheat sheet

```
"#login-btn"               /* id="login-btn" */
".card.active"              /* class="card active" (both classes) */
"input[type='email']"       /* attribute match */
"div.card > button"         /* direct child */
"ul li:nth-child(2)"        /* 2nd <li> in its parent */
"a[href^='https']"          /* href starts with https */
"input[placeholder*='mail']" /* placeholder contains "mail" */
"form input, form select"   /* multiple selectors (comma) */
```

3.3 XPath cheat sheet

```
"//button"                          # all <button> elements, anywhere
"//div[@id='main']"                 # div with a specific id
"//button[text()='Submit']"         # exact text match
"//button[contains(text(),'Sub')]"  # partial text match
"//input[@type='text' and @name='email']" # multiple attribute conditions
"//label[text()='Email']/following-sibling::input" # sibling traversal
"//div[@class='card']/parent::div"  # move UP to a parent
"//ul/li[last()]"                   # last list item
"//*[@data-testid='submit-btn']"    # custom test attributes (recommended!)
```

TIP

Ask your developers to add stable `data-testid` (or `data-qa`) attributes to key elements. Unlike CSS classes or generated ids, these are added specifically for tests, rarely change during refactors, and make locators extremely resilient: `By.CSS_SELECTOR, "[data-testid='login-button']"`.

3.4 Chaining locators (finding elements inside elements)

```
form = driver.find_element(By.ID, "checkout-form")
email_field = form.find_element(By.NAME, "email") # search scoped inside ‘form‘
all_inputs = form.find_elements(By.TAG_NAME, "input")
```

RETURNS

Calling `.find_element()` / `.find_elements()` on a `WebElement` (instead of on `driver`) restricts the search to descendants of that element. This is called **relative** or **scoped** location, and it is a key technique for writing precise, non-ambiguous locators in complex pages with repeated structures (e.g. picking "the delete button inside row 3", not any delete button on the page).

3.5 Inspecting a `WebElement`

Once you have a `WebElement`, it exposes rich information and behaviour:

```
element.text                    # -> str: visible, rendered inner text
element.get_attribute("value")  # -> str or None: HTML attribute / DOM property
element.get_dom_attribute("class") # -> str or None: raw HTML attribute only
element.tag_name                # -> str: e.g. "input", "div"
element.is_displayed()          # -> bool: is it visible on the page?
element.is_enabled()            # -> bool: can the user interact with it?
element.is_selected()           # -> bool: for checkboxes/radios/options
element.size                    # -> dict: {'height': .., 'width': ..}
element.location                # -> dict: {'x': .., 'y': ..} top-left corner
element.rect                    # -> dict: combines location + size
element.value_of_css_property("color") # -> str: computed CSS value
```

COMMON PITFALL

`element.text` only returns text that is **actually rendered and visible** in the browser (it respects CSS such as `display:none`). If you need the raw text regardless of visibility, use JavaScript execution: `driver.execute_script("return arguments[0].textContent;", element)`.

Chapter 4

Waiting Strategies

4.1 Why waits matter

Modern web pages load content asynchronously: JavaScript fetches data, renders components, and updates the DOM *after* the initial page load. If your script tries to interact with an element before it exists or before it becomes interactable, Selenium throws exceptions such as `NoSuchElementException` or `ElementNotInteractableException`.

CONCEPT

There are three waiting strategies in Selenium. Understanding the difference – and never mixing implicit and explicit waits carelessly – is one of the most important skills for a reliable, non-flaky test suite.

4.2 Hard-coded sleeps (avoid)

```
import time
time.sleep(5)  # AVOID in real test code
```

COMMON PITFALL

`time.sleep()` always waits the full duration, even if the element appeared after 200ms – wasting time across thousands of test runs. Worse, if the element takes longer than the fixed sleep, the test still fails intermittently ("flaky tests"). Never use hard sleeps for synchronization; only use them for very rare, deliberate scenarios (e.g. simulating a slow user, and even then prefer waits).

4.3 Implicit waits

```
driver.implicitly_wait(10) # applies for the ENTIRE life of the driver session
```

An implicit wait tells the driver: "whenever `find_element`/`find_elements` doesn't find a match immediately, keep polling for up to N seconds before giving up." It is set once and silently applies to every future locator call on that driver.

COMMON PITFALL

Implicit waits only help with element *presence in the DOM* – not with visibility, clickability, or text content. They can also interact badly with explicit waits (mixing the two can cause unpredictable total wait times). Most teams today set a small implicit wait (or none at all) and rely on explicit waits for real synchronization logic.

4.4 Explicit waits – `WebDriverWait`

Explicit waits are the recommended approach: you wait for a *specific, named condition* to become true, with a defined timeout and polling frequency.

```
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
from selenium.webdriver.common.by import By

wait = WebDriverWait(driver, timeout=10, poll_frequency=0.5)

element = wait.until(
    EC.presence_of_element_located((By.ID, "results"))
)
```

RETURNS

`WebDriverWait(driver, 10).until(condition)` repeatedly calls `condition(driver)` every `poll_frequency` seconds (default 0.5s) until it returns a truthy value, **or** the timeout expires. On success it returns *whatever the condition function returned* (often a `WebElement`, sometimes a `bool`). On timeout it raises `selenium.common.exceptions.TimeoutException`.

4.5 Common `expected_conditions` (EC)

```
EC.presence_of_element_located((By.ID, "x"))
# -> WebElement once it exists in the DOM (may still be invisible)

EC.visibility_of_element_located((By.ID, "x"))
# -> WebElement once it exists AND has non-zero size / is displayed

EC.invisibility_of_element_located((By.ID, "loading-spinner"))
# -> True once the element is gone or hidden (great for waiting out spinners)

EC.element_to_be_clickable((By.ID, "submit"))
# -> WebElement once visible AND enabled (ready to receive a click)

EC.text_to_be_present_in_element((By.ID, "status"), "Success")
# -> bool: True once the element's text contains the given substring

EC.title_is("Dashboard")         # -> bool
EC.title_contains("Dash")        # -> bool
EC.url_to_be("https://x.com/done") # -> bool
EC.number_of_windows_to_be(2)    # -> bool
EC.alert_is_present()            # -> Alert object once a JS alert appears
EC.staleness_of(old_element)     # -> bool: True once old_element is detached from DOM
```

TIP

Use `EC.element_to_be_clickable` before clicking anything important. It combines two checks (`is_displayed()` and `is_enabled()`) that together prevent the two most common click-related flaky failures.

4.6 Writing a custom wait condition

Sometimes the built-in conditions aren't enough. Any callable that accepts `driver` and returns a truthy value works:

```
def cart_has_items(expected_count):
    def _predicate(driver):
        items = driver.find_elements(By.CSS_SELECTOR, ".cart-item")
        return len(items) == expected_count and items or False
    return _predicate

wait.until(cart_has_items(3))
```

4.7 Fluent Wait (fine-grained control)

```
from selenium.common.exceptions import NoSuchElementException

wait = WebDriverWait(
    driver,
    timeout=20,
    poll_frequency=1,
    ignored_exceptions=[NoSuchElementException]
)
element = wait.until(EC.presence_of_element_located((By.ID, "late-element")))
```

`ignored_exceptions` tells the wait to swallow specific exceptions raised *during polling* rather than failing immediately – useful for elements that briefly throw `StaleElementReferenceException` while the page is still re-rendering.

NOTE

Rule of thumb: use *one* synchronization strategy consistently across your framework – almost always explicit waits (`WebDriverWait` + `expected_conditions`). Avoid setting a nonzero `implicitly_wait()` on the same driver that also uses explicit waits.

Chapter 5

Interacting with Elements

5.1 Basic interaction methods

```
element.click()              # simulates a mouse click
element.send_keys("hello")   # types text into a field (appends to existing text)
element.clear()              # clears text from an <input> or <textarea>
element.submit()             # submits the containing <form>
```

RETURNS

All four methods above return `None`. They are commands, not queries – their purpose is the side effect on the browser. To verify the outcome, inspect element state *afterward* (e.g. read `element.get_attribute("value")` after `send_keys`, or check that a new page/element appeared after `click`).

COMMON PITFALL

`send_keys()` does **not** clear existing content first – it appends. If a field might already contain text (e.g. re-running a test, or a pre-filled default), call `element.clear()` immediately before `send_keys()`.

5.2 Sending special keys

```
from selenium.webdriver.common.keys import Keys

search_box.send_keys("selenium python")
search_box.send_keys(Keys.ENTER)      # press Enter
search_box.send_keys(Keys.TAB)        # move focus to next field
search_box.send_keys(Keys.CONTROL, "a") # Ctrl+A (select all)
search_box.send_keys(Keys.ESCAPE)

# Combining keys in one call:
search_box.send_keys(Keys.CONTROL + "a") # string concatenation also works
```

5.3 Working with dropdowns – the `Select` class

Native HTML `<select>` dropdowns need a dedicated helper class – clicking on the `<option>` elements directly is unreliable across browsers.

```
from selenium.webdriver.support.ui import Select

dropdown = Select(driver.find_element(By.ID, "country"))

dropdown.select_by_visible_text("Germany")
dropdown.select_by_value("DE")
dropdown.select_by_index(2)

# Reading state
dropdown.first_selected_option    # -> WebElement of the currently selected option
dropdown.options                  # -> list[WebElement] of ALL <option> elements
dropdown.all_selected_options     # -> list[WebElement], for multi-selects

# Multi-select dropdowns only:
dropdown.deselect_all()
dropdown.deselect_by_value("DE")
```

COMMON PITFALL

`Select` only works on genuine `<select>` HTML elements. Many modern UIs (React, Angular, custom design systems) build "fake" dropdowns out of `<div>`s and `<li>`s. For those, you must click the dropdown to open it, then locate and click the desired option like any other elements – `Select` will raise `UnexpectedTagNameException` if used on the wrong element type.

5.4 Checkboxes and radio buttons

```
checkbox = driver.find_element(By.ID, "subscribe")

if not checkbox.is_selected():
    checkbox.click()        # toggle ON only if not already checked

radio = driver.find_element(By.CSS_SELECTOR, "input[value='premium']")
radio.click()
assert radio.is_selected() is True
```

5.5 File uploads

For standard HTML `<input type="file">` elements, you do not need a native OS dialog handler – simply send the absolute file path as text:

```
upload_input = driver.find_element(By.CSS_SELECTOR, "input[type='file']")
upload_input.send_keys("/home/user/documents/report.pdf")
```

TIP

This trick works because `send_keys` on a file input sets its value directly through the WebDriver protocol, bypassing the OS file picker entirely – no need for `pyautogui` or similar tools for this common case.

Chapter 6

ActionChains: Advanced Mouse and Keyboard Control

6.1 What is `ActionChains`?

CONCEPT

Basic methods like `element.click()` or `element.send_keys()` perform *one atomic user gesture at a time*, and Selenium executes each command immediately. But real users often perform **compound gestures**: hover then click, drag-and-drop, right-click for a context menu, click-and-hold, or a specific sequence of key presses combined with mouse movement.
`ActionChains` is a builder class that lets you **queue up** a sequence of low-level mouse and keyboard actions, and then execute them all together, in order, with a single call to `.perform()`. It models input at a lower level than the basic element methods – closer to how the operating system reports mouse movement and key events.

```
from selenium.webdriver import ActionChains

actions = ActionChains(driver)
actions.move_to_element(menu).click(submenu_item).perform()
```

RETURNS

Every chainable method on `ActionChains` (`.click()`, `.move_to_element()`, `.key_down()`, etc.) returns the **same `ActionChains` instance** (`self`), which is what enables the fluent, dot-chained syntax. **Nothing actually happens in the browser until `.perform()` is called** – until then, you are only building up a queue of pending actions. `.perform()` itself returns `None`.

COMMON PITFALL

Forgetting the trailing `.perform()` is one of the most common beginner mistakes with `ActionChains`: the code runs without errors, but nothing happens on screen, because the queued actions were built but never executed.

6.2 Hovering (mouse-over menus)

```
menu = driver.find_element(By.ID, "nav-products")
submenu_item = driver.find_element(By.LINK_TEXT, "Laptops")
```

```
ActionChains(driver)\
    .move_to_element(menu)\
    .pause(0.5)\
    .click(submenu_item)\
    .perform()
```

`move_to_element` moves the virtual mouse cursor to the center of an element – essential for CSS `:hover`-triggered dropdown menus that never appear from a plain `.click()`.

6.3 Drag and drop

```
source = driver.find_element(By.ID, "draggable-item")
target = driver.find_element(By.ID, "drop-zone")

# Simple, single-step drag and drop
ActionChains(driver).drag_and_drop(source, target).perform()

# More reliable for JS-heavy drag/drop widgets: break it into explicit steps
ActionChains(driver)\
    .click_and_hold(source)\
    .move_to_element(target)\
    .pause(0.3)\
    .release()\
    .perform()

# Drag by a pixel offset instead of to a specific element
ActionChains(driver).drag_and_drop_by_offset(source, xoffset=100, yoffset=0).perform()
```

TIP

Many modern drag-and-drop widgets (built with libraries like SortableJS or react-dnd) listen for granular `mousemove` events and don't respond correctly to the single `drag_and_drop()` shortcut. If drag-and-drop tests are flaky, switch to the explicit `click_and_hold` → `move_to_element` → `pause` → `release` sequence, sometimes adding several intermediate `move_by_offset()` calls to simulate a smoother real drag.

6.4 Right-click (context menu) and double-click

```
element = driver.find_element(By.ID, "file-icon")

ActionChains(driver).context_click(element).perform() # right-click
ActionChains(driver).double_click(element).perform() # double-click
```

6.5 Keyboard modifiers combined with mouse actions

```
from selenium.webdriver.common.keys import Keys

first_item = driver.find_element(By.CSS_SELECTOR, ".list-item:nth-child(1)")
third_item = driver.find_element(By.CSS_SELECTOR, ".list-item:nth-child(3)")
```

```

# Select multiple items with Ctrl held down (like a file manager)
ActionChains(driver)\
    .key_down(Keys.CONTROL)\
    .click(first_item)\
    .click(third_item)\
    .key_up(Keys.CONTROL)\
    .perform()
```

`key_down` presses a key *and holds it* (without releasing), `key_up` releases it. This is essential for shortcuts like Ctrl+Click, Shift+Click, or Ctrl+C/Ctrl+V that require the modifier key to remain pressed while another action occurs.

6.6 Full `ActionChains` method reference

Method	Effect
`.click(element=None)`	Click; if no element given, clicks at the current cursor position.
`.click_and_hold(element=None)`	Press and hold the left mouse button.
`.release(element=None)`	Release a held mouse button.
`.context_click(element=None)`	Right-click.
`.double_click(element=None)`	Double left-click.
`.move_to_element(element)`	Move mouse to the element's center.
`.move_to_element_with_offset(elem, x, y)`	Move to a point offset from the element's top-left corner.
`.move_by_offset(x, y)`	Move the mouse by a relative pixel offset from its current position.
`.drag_and_drop(source, target)`	Click-hold source, move to target, release.
`.drag_and_drop_by_offset(src, x, y)`	Drag by a relative pixel offset.
`.send_keys(*keys)`	Send keystrokes to whichever element currently has focus.
`.send_keys_to_element(element, *keys)`	Click the element first, then send keystrokes.
`.key_down(key)` / `.key_up(key)`	Press / release a modifier key without an accompanying click.
`.pause(seconds)`	Insert a deliberate pause between queued actions.
`.perform()`	**Execute** every queued action, in order.
`.reset_actions()`	Clear the queue and release any currently-held keys/buttons.

NOTE

Every `ActionChains` object accumulates its queue across calls until `.perform()` runs. If you reuse the same `ActionChains` instance for a second, unrelated sequence, call `.reset_actions()` first, or simply create a fresh `ActionChains(driver)` instance – otherwise old queued actions may replay unexpectedly.

Chapter 7

Frames, Alerts, and Multi-Window Handling

7.1 IFrames

CONCEPT

An `<iframe>` embeds an entirely separate HTML document inside the current page. Selenium's default search scope is the *top-level* document – it cannot "see" elements inside an iframe until you explicitly switch context into that frame.

```
# Switch by index (0 = first iframe on the page), by name/id, or by WebElement
driver.switch_to.frame(0)
driver.switch_to.frame("payment-frame")
driver.switch_to.frame(driver.find_element(By.CSS_SELECTOR, "iframe.checkout"))

# ... now find_element/find_elements search INSIDE the iframe ...
card_number_field = driver.find_element(By.ID, "card-number")
card_number_field.send_keys("4242424242424242")

# Return to the main/top-level document
driver.switch_to.default_content()

# From a nested frame, go up exactly one level (to the parent frame)
driver.switch_to.parent_frame()
```

COMMON PITFALL

If you call `find_element` on an element that lives inside an iframe *without* switching into that frame first, Selenium will raise `NoSuchElementException` even though the element is technically "on the page" – because Selenium's scope did not include the frame's document.

7.2 JavaScript alerts, confirms, and prompts

Native browser dialogs (`alert()`, `confirm()`, `prompt()`) are **not part of the DOM** and cannot be located with `find_element`. They are handled via `driver.switch_to.alert`.

```
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
```

```
trigger_button.click() # some action that pops up a JS dialog

alert = WebDriverWait(driver, 10).until(EC.alert_is_present())

alert.text                # -> str: the dialog's message
alert.accept()             # click "OK"
alert.dismiss()            # click "Cancel"
alert.send_keys("my input") # type into a prompt() dialog before accepting
```

RETURNS

`driver.switch_to.alert` returns an `Alert` object. Unlike `WebElement`, it exposes only four members: `.text`, `.accept()`, `.dismiss()`, and `.send_keys()`. Attempting any DOM interaction with it (like `.click()`) will fail – it is not a `WebElement`.

7.3 Multiple windows and tabs – revisited

See Section 2.5 for the full pattern. As a quick summary: enumerate `driver.window_handles`, use `driver.switch_to.window(handle)` to change focus, and remember every `find_element` call operates against whichever window/tab currently has focus.

Chapter 8

JavaScript Execution, Screenshots, and Cookies

8.1 Executing JavaScript directly

Sometimes the WebDriver API alone can't achieve what you need (scrolling precisely, reading a JS variable, bypassing a flaky animation). `execute_script` runs arbitrary JavaScript inside the browser, in the context of the current page.

```
# Scroll an element into view
driver.execute_script("arguments[0].scrollIntoView(true);", element)

# Read a value computed by JavaScript
page_height = driver.execute_script("return document.body.scrollHeight;")

# Set an element's value directly (bypassing native events -- use sparingly)
driver.execute_script("arguments[0].value = arguments[1];", input_field, "test@example.
    com")

# Click via JS when a normal .click() is intercepted by an overlay
driver.execute_script("arguments[0].click();", element)
```

RETURNS

`execute_script(script, *args)` returns whatever the JavaScript `return` statement produces, automatically converted to a Python type: JS `null`/`undefined` → `None`, numbers → `int`/`float`, strings → `str`, arrays → `list`, objects → `dict`, and DOM elements → `WebElement`. If the script has no `return`, the method returns `None`.

`arguments[0]`, `arguments[1]`, etc. inside the script string refer to the Python arguments you pass after the script (in order) – this is how you pass `WebElement` references and other values into the JS execution context.

COMMON PITFALL

Using `execute_script` to click elements or set field values bypasses the real user-interaction event chain (`mousedown`, `focus`, `input`, etc.), which some JavaScript frameworks depend on to update their internal state. Treat it as an escape hatch for otherwise-unworkable situations, not a default replacement for `.click()` / `.send_keys()`.

8.2 Asynchronous scripts

```
result = driver.execute_async_script(
    """
    var callback = arguments[arguments.length - 1];
    fetch('/api/status').then(r => r.json()).then(data => callback(data));
    """
)
```

`execute_async_script` is for JavaScript that completes asynchronously (promises, callbacks, timers). The script must invoke the special final `arguments[arguments.length - 1]` callback with its result; Selenium waits (up to `set_script_timeout()`) for that callback to fire.

8.3 Taking screenshots

```
driver.save_screenshot("homepage.png")       # -> bool: True on success
png_bytes = driver.get_screenshot_as_png()   # -> bytes
base64_str = driver.get_screenshot_as_base64() # -> str

# Screenshot of a single element (very useful for visual diffs / evidence)
element.screenshot("submit-button.png")      # -> bool
```

TIP

Capture a screenshot automatically whenever a test fails – this is one of the highest-value habits for any automation framework. In `pytest`, this is commonly implemented as a fixture teardown hook that checks the test outcome (see Chapter 11).

8.4 Cookies

```
driver.get_cookies()                    # -> list[dict]: all cookies for current domain
driver.get_cookie("session_id")         # -> dict or None: one specific cookie
driver.add_cookie({"name": "theme", "value": "dark"})
driver.delete_cookie("theme")
driver.delete_all_cookies()
```

RETURNS

Each cookie is represented as a `dict` with keys such as `name`, `value`, `domain`, `path`, `expiry`, `secure`, and `httpOnly`. A common pattern for speeding up test suites is to log in once via the UI, capture the session cookie with `get_cookies()`, and then `add_cookie()` it directly in later tests to skip repeating the login flow – **note** you must first navigate to the target domain with `driver.get()` before `add_cookie()` will work, since cookies are domain-scoped.

Chapter 9

Exceptions and Robust Error Handling

9.1 Common exceptions you will meet

Exception	Typical cause
`NoSuchElementException`	Locator matched nothing (wrong locator, element not yet rendered, or wrong frame/window).
`TimeoutException`	An explicit `WebDriverWait` condition never became true within the timeout.
`ElementNotInteractableException`	Element exists but cannot receive the action (e.g. hidden, zero size, off-screen).
`ElementClickInterceptedException`	Another element (an overlay, modal, sticky header) is visually on top of the target.
`StaleElementReferenceException`	The element was found, but the DOM node it points to has since been removed/replaced (common after page updates or re-renders).
`InvalidSelectorException`	The locator string itself is syntactically invalid (bad CSS/XPath).
`UnexpectedAlertPresentException`	A JS alert popped up and is blocking other commands.
`WebDriverException`	Base class for driver/session-level failures (e.g. browser crashed, session lost).

CONCEPT

`StaleElementReferenceException` **deep dive.** A `WebElement` in Selenium is really a *reference* (an internal ID) to a specific DOM node at the moment it was found. If the page later re-renders that portion of the DOM – even to display what looks like "the exact same" element – the old reference becomes invalid ("stale"), and Selenium will refuse to interact with it. **The fix is to re-find the element** after any action that could trigger a re-render (AJAX refresh, React re-render, page navigation), rather than reusing an old reference.

9.2 Import path for exceptions

```
from selenium.common.exceptions import (
    NoSuchElementException,
    TimeoutException,
    StaleElementReferenceException,
```

```
    ElementClickInterceptedException,
    ElementNotInteractableException,
)
```

9.3 Retry pattern for stale elements

```python
from selenium.common.exceptions import StaleElementReferenceException

def click_with_retry(locate_fn, attempts=3):
    """locate_fn: a zero-arg callable that re-finds and returns the element."""
    for attempt in range(attempts):
        try:
            element = locate_fn()
            element.click()
            return
        except StaleElementReferenceException:
            if attempt == attempts - 1:
                raise
            continue

click_with_retry(lambda: driver.find_element(By.ID, "refresh-row-btn"))
```

9.4 Handling intercepted clicks

```python
from selenium.common.exceptions import ElementClickInterceptedException

try:
    submit_button.click()
except ElementClickInterceptedException:
    # Common causes: a cookie-consent banner, a modal, or a sticky nav bar
    driver.execute_script("arguments[0].scrollIntoView({block:'center'});", submit_button
        )
    submit_button.click()
```

TIP

Before adding retry/exception-handling logic, first ask *why* the failure happens. Often the real fix is a better wait condition (e.g. waiting for `element_to_be_clickable` instead of just `presence_of_element_located`), not a try/except band-aid. Reach for exception handling for genuinely unpredictable timing issues, not as a substitute for correct synchronization.

Chapter 10

Structuring Test Code: The Page Object Model

10.1 The problem POM solves

Without structure, test scripts accumulate raw locators and interaction code directly inside test functions. When the UI changes (an id is renamed, a button moves), you must hunt through every test file that touched that element. The **Page Object Model (POM)** solves this by concentrating all knowledge of a page's structure into one dedicated class.

CONCEPT

Page Object Model: one class per page (or major component) of the application. The class owns:

- all locators for that page,
- methods representing user-facing actions (`login()`, `add_to_cart()`, `search_for(query)`),
- methods for reading page state relevant to assertions.

Test functions then read like a script of business actions, entirely decoupled from raw Selenium calls and CSS/XPath details.

10.2 A basic Page Object

```python
# pages/login_page.py
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC


class LoginPage:
    URL = "https://example.com/login"

    USERNAME_INPUT = (By.ID, "username")
    PASSWORD_INPUT = (By.ID, "password")
    SUBMIT_BUTTON = (By.CSS_SELECTOR, "button[type='submit']")
    ERROR_MESSAGE = (By.CSS_SELECTOR, ".alert-error")

    def __init__(self, driver):
        self.driver = driver
        self.wait = WebDriverWait(driver, 10)
```

```
18
19      def open(self):
20          self.driver.get(self.URL)
21          return self
22
23      def login(self, username, password):
24          self.wait.until(EC.visibility_of_element_located(self.USERNAME_INPUT))
25          self.driver.find_element(*self.USERNAME_INPUT).send_keys(username)
26          self.driver.find_element(*self.PASSWORD_INPUT).send_keys(password)
27          self.driver.find_element(*self.SUBMIT_BUTTON).click()
28
29      def get_error_message(self):
30          element = self.wait.until(EC.visibility_of_element_located(self.ERROR_MESSAGE))
31          return element.text
```

NOTE

Notice locators are stored as `(By.X, "value")` tuples, then unpacked with the `*` operator: `driver.find_element(*self.USERNAME_INPUT)` is equivalent to `driver.find_element(By.ID, "username")`. This keeps every locator declared once, at the top of the class, easy to find and update.

10.3 Using the Page Object from a test

```
1  # tests/test_login.py
2  from pages.login_page import LoginPage
3
4  def test_invalid_login_shows_error(driver):
5      login_page = LoginPage(driver).open()
6      login_page.login("wrong_user", "wrong_password")
7
8      assert login_page.get_error_message() == "Invalid username or password"
```

The test itself contains zero locators and zero raw Selenium calls – if the login form's HTML structure changes, only `LoginPage` needs updating.

10.4 Base Page pattern (shared behaviour)

For larger suites, extract common functionality (waits, navigation helpers) into a shared base class that every page object inherits from:

```
1  class BasePage:
2      def __init__(self, driver):
3          self.driver = driver
4          self.wait = WebDriverWait(driver, 10)
5
6      def wait_for_visible(self, locator):
7          return self.wait.until(EC.visibility_of_element_located(locator))
8
9      def wait_for_clickable(self, locator):
10         return self.wait.until(EC.element_to_be_clickable(locator))
11
12
```

```
class LoginPage(BasePage):
    USERNAME_INPUT = (By.ID, "username")
    # ...
    def login(self, username, password):
        self.wait_for_visible(self.USERNAME_INPUT).send_keys(username)
        # ...
```

Chapter 11

Integrating with Pytest

11.1 Why pytest over raw scripts or unittest

`pytest` is the de facto standard Python test runner: concise syntax (plain `assert` statements, no `self.assertEqual`), a powerful **fixture** system for setup/teardown, built-in parametrization, and a huge plugin ecosystem (`pytest-html`, `pytest-xdist` for parallelism, `pytest-rerunfailures`).

11.2 The driver fixture

A **fixture** is a reusable setup/teardown function that pytest injects into any test that requests it by name (as a parameter). This is the standard, robust way to manage the WebDriver lifecycle.

```python
# conftest.py (shared fixtures, auto-discovered by pytest)
import pytest
from selenium import webdriver
from selenium.webdriver.chrome.options import Options


@pytest.fixture
def driver():
    options = Options()
    options.add_argument("--start-maximized")
    drv = webdriver.Chrome(options=options)

    yield drv              # <-- everything before yield = setup

    drv.quit()              # <-- everything after yield = teardown
```

CONCEPT

`yield` splits the fixture into two halves. Code *before* `yield` runs once before the test body executes; the `yield`ded value (`drv`) is passed into the test function as the argument named `driver`; code *after* `yield` runs after the test finishes – **even if the test fails or raises an exception** – guaranteeing `driver.quit()` always executes and no browser sessions leak.

```python
# test_search.py
def test_search_returns_results(driver):
    driver.get("https://example.com")
    driver.find_element(By.NAME, "q").send_keys("selenium")
```

```
    driver.find_element(By.NAME, "q").submit()

    results = driver.find_elements(By.CSS_SELECTOR, ".result-item")
    assert len(results) > 0
```

11.3 Fixture scope

```
@pytest.fixture(scope="function") # default: new instance per test (safest, isolated)
@pytest.fixture(scope="class")   # shared across all tests in one test class
@pytest.fixture(scope="module")  # shared across all tests in one file
@pytest.fixture(scope="session") # shared across the ENTIRE test run
```

COMMON PITFALL

Sharing a single browser session across many tests (`scope="session"`) speeds up a suite, but breaks test isolation: leftover cookies, localStorage, or page state from one test can silently affect the next, and a crash in one test can take down every subsequent test sharing that driver. Default to `scope="function"` unless you have a specific, well-understood reason (and a cleanup strategy) for sharing.

11.4 Automatic screenshot-on-failure

```
# conftest.py
import pytest

@pytest.hookimpl(tryfirst=True, hookwrapper=True)
def pytest_runtest_makereport(item, call):
    outcome = yield
    report = outcome.get_result()
    setattr(item, f"rep_{report.when}", report)


@pytest.fixture
def driver(request):
    drv = webdriver.Chrome()
    yield drv

    # After the test: check if it failed, and if so, capture evidence
    if request.node.rep_call.failed:
        drv.save_screenshot(f"failure_{request.node.name}.png")

    drv.quit()
```

11.5 Parametrizing tests (e.g. cross-browser or data-driven runs)

```
import pytest
from selenium import webdriver

@pytest.fixture(params=["chrome", "firefox"])
def driver(request):
```

```
    if request.param == "chrome":
        drv = webdriver.Chrome()
    else:
        drv = webdriver.Firefox()
    yield drv
    drv.quit()


@pytest.mark.parametrize("username,password,expected_error", [
    ("", "pass123", "Username is required"),
    ("user", "", "Password is required"),
    ("baduser", "badpass", "Invalid username or password"),
])
def test_login_validation(driver, username, password, expected_error):
    login_page = LoginPage(driver).open()
    login_page.login(username, password)
    assert login_page.get_error_message() == expected_error
```

RETURNS

`@pytest.fixture(params=[...])` causes pytest to run *every test that uses this fixture once per parameter value* – here, once against Chrome and once against Firefox, automatically. `@pytest.mark.parametrize` does the same for explicit argument sets on a single test function, generating one test case per tuple in the list (here, 3 separate test runs, each with a distinct, readable ID in the test report).

11.6 Running and reporting

```
pytest                                  # run all tests
pytest test_login.py                    # run one file
pytest -k "login"                       # run tests matching a keyword
pytest -m smoke                         # run tests tagged @pytest.mark.smoke
pytest --html=report.html --self-contained-html # generate an HTML report
pytest -n 4                              # run in parallel (needs pytest-xdist)
pytest --reruns 2                        # retry failed tests (needs pytest-rerunfailures)
```

Chapter 12

Best Practices and Common Pitfalls

12.1 Locator strategy

- Prefer stable attributes: `id`, `data-testid` > `name` > CSS selector > XPath.
- Never rely on absolute XPath generated by "copy XPath" browser tools (e.g. `/html/body/div[3]/div/div[2]/`) – it breaks with the smallest layout change. Write relative, attribute-based locators instead.
- Keep every locator in one place (Page Objects), never scattered across test files.

12.2 Synchronization

- Standardize on explicit waits (`WebDriverWait` + `expected_conditions`) across the whole framework.
- Never use `time.sleep()` for synchronization; reserve it (rarely) for simulating deliberate delays unrelated to element state.
- Wait for `element_to_be_clickable` before any click that matters to the test outcome.

12.3 Test design

- Each test should be independent and able to run in any order or in isolation – never depend on state left behind by a previous test.
- Keep assertions focused: one logical behaviour per test makes failures easy to diagnose.
- Use the Page Object Model (or a similar abstraction layer) from day one, even for small suites – retrofitting it later is expensive.
- Separate test *data* from test *logic* (e.g. via fixtures, JSON/YAML files, or parametrize) so non-developers can update data without touching code.

12.4 Performance and stability

- Run suites headless in CI for speed; validate periodically in headed mode.
- Isolate each test's browser state (fresh driver instance, or clear cookies/localStorage between tests) to avoid cross-test contamination.
- Capture a screenshot (and ideally the page source) automatically on every failure – this alone eliminates most "why did this fail?" investigation time.

- Avoid tests that depend on exact pixel positions, animation timing, or network response duration; wait for functional conditions instead.

12.5 Maintainability

- Name tests descriptively: `test_checkout_fails_with_expired_card`, not `test_1`.
- Log meaningfully – when a test fails at 3am in CI, logs (not just a stack trace) are what let someone diagnose it without reproducing locally.
- Version-pin your `selenium` package and browser/driver versions in CI to avoid surprise breakages from upstream updates.
- Review and refactor Page Objects as the application evolves; treat them as production code, with code review, not throwaway scripts.

COMMON PITFALL

The #1 cause of "flaky" Selenium suites is inconsistent or missing synchronization – mixing implicit and explicit waits, using `time.sleep()` inconsistently, or clicking elements without confirming they are visible and enabled first. If a suite is flaky, audit your waiting strategy before anything else.

Chapter 13

Quick Reference Cheat Sheet

13.1 Setup

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.common.keys import Keys
from selenium.webdriver.support.ui import WebDriverWait, Select
from selenium.webdriver.support import expected_conditions as EC
from selenium.webdriver import ActionChains
from selenium.common.exceptions import (
    NoSuchElementException, TimeoutException, StaleElementReferenceException
)

driver = webdriver.Chrome()
wait = WebDriverWait(driver, 10)
```

13.2 One-page summary table

Call	Returns	Notes
`webdriver.Chrome()`	`WebDriver`	Starts a browser session
`driver.get(url)`	`None`	Navigates and waits for load
`driver.find_element(by, val)`	`WebElement`	Raises if not found
`driver.find_elements(by, val)`	`list`	Empty list if none found
`element.click()`	`None`	Simulated mouse click
`element.send_keys(text)`	`None`	Appends, does not clear first
`element.clear()`	`None`	Empties text field
`element.text`	`str`	Visible rendered text only
`element.get_attribute(name)`	`str/None`	HTML attribute or DOM property
`element.is_displayed()`	`bool`	Visibility check
`element.is_enabled()`	`bool`	Interactable check
`WebDriverWait(driver, n).until(cond)`	varies	Depends on the condition used
`Select(element)`	`Select`	Wraps a native `<select>`

Call	Returns	Notes
`ActionChains(driver)...perform()`	`None`	Executes the queued gesture sequence
`driver.switch_to.frame(x)`	`None`	Enters an iframe's context
`driver.switch_to.alert`	`Alert`	Handles native JS dialogs
`driver.execute_script(js, *args)`	varies	Return value mirrors the JS `return`
`driver.get_screenshot_as_png()`	`bytes`	Raw PNG image data
`driver.get_cookies()`	`list[dict]`	All cookies for current domain
`driver.quit()`	`None`	Ends the entire session

13.3 Most-used `expected_conditions`

```
EC.presence_of_element_located((By.ID, "x"))
EC.visibility_of_element_located((By.ID, "x"))
EC.invisibility_of_element_located((By.ID, "x"))
EC.element_to_be_clickable((By.ID, "x"))
EC.text_to_be_present_in_element((By.ID, "x"), "text")
EC.alert_is_present()
EC.staleness_of(element)
EC.number_of_windows_to_be(2)
```

13.4 Glossary

Term	Meaning
WebDriver	The W3C protocol/API used to programmatically control a browser; also the name of the core class instance representing one browser session.
Driver executable	Browser-specific binary (`chromedriver`, `geckodriver`) that translates WebDriver commands into native browser actions.
WebElement	Python object representing one specific HTML element found on the page.
Locator	A (strategy, value) pair – e.g. `(By.ID, "email")` – used to find one or more elements.
Explicit wait	A targeted wait for a specific, named condition (via `WebDriverWait`), with its own timeout.
Implicit wait	A blanket, driver-wide polling timeout applied to every `find_element` call.
ActionChains	A builder for queuing and executing compound, low-level mouse/keyboard gesture sequences.
Stale element	A `WebElement` reference pointing to a DOM node that has since been removed or replaced.
Headless	Running a browser without a visible graphical window.
Page Object Model	Design pattern that encapsulates a page's locators and actions inside a dedicated class.
Fixture (pytest)	A reusable setup/teardown routine injected into tests by name.

Term	Meaning
Flaky test	A test that passes and fails intermittently without any code change, usually due to timing/synchronization issues.

End of document – keep this guide updated as your framework and the Selenium API evolve.